Loammi Code Tutorial - Python and R

This document will help to explain what each code does, how to use it, and what the results mean. There are comments throughout the notebooks as well to help guide you through the analysis. Sample .csv data from the study is included, as well as relevant code outputs, but you can also use your own data as long as it matches the file formatting. Due to their large size, GeoTIFF rasters and shapefiles are NOT included; but you can download the ones we used at the following links:

Florida Cooperative Land Cover Layer:
https://myfwc.com/research/gis/wildlife/cooperative-land-cover/

FNAI Conservation Lands:
https://www.fnai.org/publications/gis-data

For any questions or concerns, please reach out to avacjavacj@gmail.com.

1. LoammiOccs.R

This is the code used to generate our cleaned, filtered GPS point layers of *A. loammi* observations, the random bias background layer, and the weighted bias background layer for the land cover index calculation. For the purposes of this paper, only the weighted bias layer was reported, and the random bias layer was a point of comparison.
If you already have a usable .csv file of observation points and do not care about bias layers, this code is totally optional. The CLCRank code will create a different index using observation points alone, although this was NOT the index used in the paper.

2. CLCRank.ipynb

To run this code, you will need:

- A .csv file of your specimen observation GPS points with latitude and longitude columns. The sample code has column names ‘decimalLatitude’ and ‘decimalLongitude’. You can either set up your .csv to have these same names and run the code as-is, or modify the code where the comment specifies updating column names to use whichever you choose.
- A GeoTIFF file in which the pixel values represent landcover types (i.e., a numeric value such as 1300). You may have to export this from a GIS software. Make sure that the file path in the input configuration links directly to the .tif file and not just the folder that it is located in!

By default, this code will:

1. Reproject GPS points to match the projection of the raster.
2. Generate a 75 meter buffer around each point to extract raster values. The buffer size can be changed if needed.
3. Calculate statistics and rankings based on both raw occurrence data and based on the index. It will output 4 .csv files:
    - landcover_counts, a file of each landcover type found within *all* the buffers and their respective proportions of the total buffer area;
    - landcover_proportions, a file that lists each *individual* buffer, its lat/long coordinates, the landcover types found within it and their respective area proportions within that buffer (this was not used in the paper but just provides data for optional analysis);
    - landcover_rankings, a file that lists each landcover type, their proportions found within total buffer area (column name 'proportion', also the same as the landcover_counts file), their area proportions of the total raster ('overall_proportion'), their ranking based on occurrence ('RankOcc', calculated by area proportion based on the total buffer areas), their ranking based on the index ('RankIndex', calculated by dividing the landcover proportions within the buffer area by their proportions in the total raster), the number of buffers in which the landcover type took the most area ('buffers_as_dominant'), the number of buffers in which the landcover type appeared ('buffer_count'), and finally, the raw pixel count of each landcover type within the buffer area and total raster area ('buffer_pixels' and 'raster_pixels', respectively).
        - NOTE: The "RankIndex" generated by this code is NOT the one used in the paper; it reflects an older analysis that has since been replaced. The new index is calculated using a separate R script, detailed later in this tutorial paper. You will also not need to run the final cell to generate the landcover_rankings file if you are using the LoammiIndex R code.
    - Loammi_Points_With_Landcover: this is an additional file that extracts the landcover type via point-based locations at the single pixel value, rather than buffering. This is just for additional analysis and was not used in the study, but may be helpful for other works.

Each cell should run back-to-back. In the main configuration cell, you only need to update the paths to your two input files, as well as setting up an output folder where the results will be stored.

3. LoammiIndex.R

To run this code, you will need to run the CLCRank code 3 times with each of the separate GPS point layers: your main set of butterfly observations (in this paper, "Loammi_Thin_150m_June"), a random bias layer, and a weighted bias layer to get the "landcover_counts" output file for each set. Ensure that you rename it each time so that it is not overwritten as you process each set.

By default, this code will calculate two indices; one using the weighted layer and one using the random layer. For the purpose of this paper, only the weighted index was used. The new index calculated with the following formula:

(Pixel count within all 117 Loammi buffers for x land cover type / total pixel count of all land cover types across 117 Loammi buffers)
%
(Pixel count within all 10,000 bg buffers for x land cover type / total pixel count of all land cover types across 10,000 bg buffers)

4. TopMaps.ipynb

To run this code, you will need to set up an initial input folder that has the following files:

- A GeoTIFF landcover raster with pixel values representing landcover types,
- A shapefile of protected lands (or whatever area you wish to extract),
- And a .csv file of landcover types which you wish to map. You have to create this file yourself, simply make a column listing all the landcover types that you want to extract from the main landcover raster (ensure that they correspond to the raster pixel values!). The code looks for a column named 'all', you may either use this when creating your .csv or modify the code where the comment mentions column name.
- You will also need to set up an output folder where the resulting maps and data will be stored.

By default, this code will:

1. Extract the total areas of the landcover types listed in your .csv file and export as a GeoTIFF file ('Top_Total.tif').
2. Extract again the protected areas of those same landcover types using the conservation lands shapefile and export as another GeoTIFF file ('Top_Protected.tif').
3. Calculate the proportions protected of each landcover type and save as a .csv file ('Top_Results.csv').

The code is designed to avoid memory issues by processing the raster in chunks, since the raster used in the study was massive (statewide Florida). Larger rasters will take a long time to process, but the code can safely be left to run in the background as you do other things.

After the initial landcover extraction, you will need to update the path in the next cell to the newly made Top_Total.tif raster so that the code only extracts the protection portions of your selected landcover types and produce a raster titled Top_Protected.tif.
Finally, you will do the same in the last cell to calculate the protected portions of each landcover type. Simply update the paths with your new Protected and Total rasters where the comments indicate you to set raster paths.

5. DistanceAnalysisBuffer.ipynb
This code assumes that you have some human land use (or other unusual) landcover types appearing within your top rankings, and you want to see how far away the specimens observed there were from another landcover type (such as their habitat, if it is nearby). You will need to create a new .csv file that lists your top landcover types, sorted in columns by whether they are 'natural' or 'human'.

To run this code, you will need:

- A .csv file of observation GPS points in lat and long columns,
- A .csv file of landcover codes sorted by 'natural' or 'human',
- And a GeoTIFF raster file with those same landcover codes as pixel values.

By default, this code will:

1. Find and print input files and .csv data (just as a sanity check).
2. Reproject GPS coordinates to match those of the GeoTIFF file and generate a 75m buffer around each point to determine the dominant landcover type.
3. For each point, list the landcover type it falls in (column name 'Point_Landcover') and calculate the distance to one of the top Natural landcover types ('Nearest_Natural' and "Distance_Meters') as listed in your initial .csv file. Distances are calculated in meters from point centers, and points that already fall within a Natural landcover type will simply have distances of 0. (Because the analysis is based on 75m buffers, there may be some points that have Point_Landcover as the same value as the Nearest_Natural column, but still have a small distance of 10/20/30 or so meters. This is simply because the buffer captured that landcover type as the dominant, but the GPS point itself fell somewhere else, i.e. a narrow road pixel!).
4. Output 2 .csv files, "all_points" and "human_land_use_points" (same analysis, but lists only the points that fell within the "human" landcover types that you designated in your initial .csv file).

6. LoammiFigures.R

This is the final code that visualizes your statistical data! Please note that these codes did NOT produce the final figures in the paper; additional editing was done in Adobe Illustrator.

To run this code, you will need:

- A .csv file of your landcover rankings (i.e. “landcover_rankings”),
    - NOTE: if you used the LoammiIndex R code for your index, you will need to create a new spreadsheet for landcover rankings manually. Ensure it has the columns “landcover_type” with the numeric code and “RankIndex” with your index values. You can use your Top_Results file from the TopMaps script for the proportions protected.
- A .csv file of the protected proportions of your top habitats (i.e. “Top_Results”),
- A .csv file of your landcover types matching codes to names (i.e. “CLCStateTable”),
- And a .csv file of your buffer distances (i.e. “all_points”).

If you have a .csv file that matches the pixel landcover codes with their actual names, the code also provides a functionality to automatically convert the numeric codes in the datasets to names before plotting. Otherwise, you can modify the code to use landcover_type pixel code values.

By default, this code will generate 4 plots:

1. Bar chart ranking of the top landcover types based on the index;
2. Bar chart of those same landcover types based on their protected portions;
3. A combined plot showing both charts side by side;
4. And a supplementary table.

You may need to manually apply a multiplier to your RankIndex column (i.e. x 10,000) before generating the 3rd plot, as it will very likely contain very tiny values. You may also need to adjust the y-axis scaling to match whatever your resulting index values are.